

SPRINT 2

TAREA 6

Eliminar una Tarea

PROYECTO INDIVIDUAL · PLANIFICADOR DE TAREAS

JavaScript · TaskManager · Arrays · Eventos · DOM · localStorage

OBJETIVO

Implementar la eliminación de tareas en el Planificador de Tareas mediante un botón en cada tarjeta, un método `deleteTask()` en `TaskManager` y un `EventListener` que actualice la interfaz y conserve los cambios en `localStorage`.

RESULTADO ESPERADO

- Cada tarea mostrada en la lista cuenta con un botón “Eliminar”.
- `TaskManager` dispone de un método `deleteTask(taskId)` que elimina la tarea correspondiente del array `this.tasks`.
- El botón “Eliminar” identifica correctamente la tarea sobre la que se hizo clic.
- Después de eliminar, las tareas se guardan nuevamente en `localStorage` y la lista se renderiza con la información actualizada.
- Al recargar la página, la tarea eliminada no vuelve a aparecer.
- El repositorio conserva los avances de todo el Sprint 2 y el README contiene el enlace actualizado al deploy.

IMPORTANTE · CIERRE DEL SPRINT 2

Esta es la última tarea del Sprint 2. Al finalizar no solo se revisará la funcionalidad de eliminación: se realizará una revisión integral del repositorio con todos los avances construidos durante el Sprint 2. El deploy deberá estar actualizado y su enlace debe aparecer en el README, siguiendo el mismo criterio de entrega utilizado en el cierre del Sprint 1.

ANTES DE COMENZAR

Continúa trabajando sobre el mismo proyecto del Planificador de Tareas. No crees un proyecto nuevo para esta tarea. La funcionalidad de eliminar debe integrarse sobre la estructura y el código que construiste en las tareas anteriores.

¿QUÉ CAMBIA EN ESTA TAREA?

ANTES	AHORA	RESULTADO
Las tareas pueden crearse y mostrarse.	Cada tarea incorpora un botón “Eliminar”.	El usuario puede iniciar una eliminación desde la interfaz.
TaskManager conserva las tareas en <code>this.tasks</code> .	<code>deleteTask(taskId)</code> filtra la tarea seleccionada.	La colección queda actualizada.
La lista se renderiza con las tareas existentes.	El click en “Eliminar” ejecuta la lógica correspondiente.	La tarjeta desaparece de la lista.
<code>localStorage</code> conserva el estado anterior.	Se ejecutan <code>save()</code> y <code>render()</code> después de eliminar.	La eliminación permanece al recargar.

ESTRUCTURA QUE DEBES CONSERVAR

```
Planificador-Tareas/  
├── css/  
│   └── styles.css  
├── js/  
│   ├── index.js  
│   └── taskManager.js  
├── index.html  
└── README.md
```

RECUERDA

La estructura puede variar si tu proyecto ya incorpora archivos adicionales, pero debes mantener organizada la separación entre HTML, estilos y JavaScript. No elimines los avances realizados en las tareas anteriores.

DESARROLLO DE LA ACTIVIDAD

PASO 1 · AÑADIR UN BOTÓN DE ELIMINAR A LA TAREA HTML

En este paso agregarás un botón visible dentro de cada tarjeta de tarea. Utilizarás la clase delete-button para poder identificarlo posteriormente desde JavaScript.

RECURSOS ÚTILES

Botones de Bootstrap · Clases CSS · Estructura HTML de las tarjetas

1. En js/taskManager.js, localiza la función createTaskHtml.
2. Revisa el HTML que devuelve esta función y ubica el lugar donde quieres presentar la acción de eliminar.
3. Agrega un elemento <button> para eliminar la tarea.
4. Asigna al botón la clase delete-button. Puedes complementar esta clase con las clases de Bootstrap que ya utiliza tu proyecto.
5. El texto visible del botón debe permitir identificar claramente la acción, por ejemplo: “Eliminar”.

```
<button class="delete-button btn btn-danger">  
  Eliminar  
</button>
```

PRUEBA EL PASO 1

1. Abre index.html en el navegador.
2. Crea una nueva tarea utilizando el formulario.
3. Ubica la nueva tarea en la lista.
4. Comprueba que la tarjeta muestra el botón “Eliminar”.
5. Todavía no es necesario que el botón elimine la tarea: en este paso estamos comprobando únicamente su presencia y su clase.

RESULTADO ESPERADO

Cada tarea creada debe mostrar un botón “Eliminar” con la clase delete-button. Si el botón no aparece, revisa createTaskHtml antes de continuar.

PASO 2 · CREAR EL MÉTODO deleteTask EN TaskManager

Ahora crearás la lógica que elimina una tarea del registro. El método recibirá el identificador de la tarea y construirá un nuevo array que conserve todas las tareas excepto la seleccionada.

Implementación

1. En js/taskManager.js, dentro de la clase TaskManager, crea un método llamado deleteTask.
2. El método debe recibir un parámetro llamado taskId.
3. Crea un array vacío llamado newTasks.
4. Recorre las tareas almacenadas en this.tasks.
5. Obtén la tarea actual del recorrido y guárdala en una variable llamada task.
6. Comprueba si task.id es diferente del taskId recibido.
7. Si los identificadores son diferentes, agrega task al array newTasks.
8. Al terminar el recorrido, asigna newTasks a this.tasks.

```
deleteTask(taskId) {  
  const newTasks = [];  
  
  for (let task of this.tasks) {  
    if (task.id !== taskId) {  
      newTasks.push(task);  
    }  
  }  
  
  this.tasks = newTasks;  
}
```

¿QUÉ ESTÁ HACIENDO ESTE MÉTODO?

Imagina que this.tasks contiene las tareas con id 1, 2 y 3, y recibimos taskId = 2. El método recorre las tres tareas, conserva la 1, descarta la 2 y conserva la 3. Finalmente, this.tasks queda compuesto por las tareas 1 y 3.

IMPORTANTE

Asegúrate de comparar valores del mismo tipo. El taskId obtenido desde un atributo data-* llega como texto, por lo que posteriormente tendrás que convertirlo a número antes de llamar a deleteTask().

PASO 3 · CONFIGURAR EL EVENTLISTENER DEL BOTÓN ELIMINAR

El método `deleteTask()` ya sabe cómo eliminar una tarea, pero todavía necesitamos conectarlo con la interacción del usuario. Para ello reutilizarás el `EventListener` del evento click asociado a la Lista de Tareas.

RECURSOS ÚTILES

`EventTarget.addEventListener()` · atributos `data-*` · `classList` · `Number()` · `save()` · `render()`

1. En `js/index.js`, localiza el `EventListener` para el evento click de la Lista de Tareas que ya utilizaste para manejar la interacción de las tarjetas.
2. Después del código que maneja el botón “Marcar como hecho”, agrega una nueva condición `if`.
3. Comprueba si `event.target.classList` contiene la clase `delete-button`.
4. Si la condición se cumple, obtén el elemento `parentTask` que representa la tarea seleccionada.
5. Obtén el valor de `data-task-id` desde `parentTask`.
6. Convierte ese valor de texto a un número.
7. Llama a `taskManager.deleteTask()` enviando el `taskId`.
8. Guarda la colección actualizada mediante `taskManager.save()`.
9. Vuelve a renderizar la lista mediante `taskManager.render()`.

```
if (event.target.classList.contains('delete-button')) {
  const parentTask = event.target.parentElement;
  const taskId = Number(parentTask.dataset.taskId);

  taskManager.deleteTask(taskId);
  taskManager.save();
  taskManager.render();
}
```

ATENCIÓN

El elemento `parentTask` debe ser el contenedor que posee el atributo `data-task-id`. Si la estructura HTML de tu tarjeta tiene más niveles, revisa qué elemento corresponde realmente a la tarea antes de obtener `dataset.taskId`.

FLUJO COMPLETO

```
Usuario hace clic en "Eliminar"
↓
Se identifica el botón delete-button
↓
Se obtiene la tarea padre
↓
Se recupera data-task-id
↓
Se convierte el id a Number
↓
taskManager.deleteTask(taskId)
↓
taskManager.save()
↓
taskManager.render()
```

PRUEBAS DE FUNCIONALIDAD

No te limites a comprobar únicamente el caso exitoso. La eliminación debe funcionar tanto en una lista con varias tareas como después de recargar la página.

CASO 1 · Eliminar una tarea

1. Abre la aplicación.
2. Crea al menos dos tareas.
3. Haz clic en “Eliminar” en una de ellas.
4. Comprueba que la tarea seleccionada desaparece de la lista.
5. Comprueba que la otra tarea permanece.

CASO 2 · Persistencia después de recargar

1. Crea dos o más tareas.
2. Elimina una tarea.
3. Recarga la página.
4. Comprueba que la tarea eliminada no vuelve a aparecer.
5. Comprueba que las demás tareas continúan disponibles.

CASO 3 · Eliminar varias tareas

1. Crea al menos tres tareas.
2. Elimina una de ellas.
3. Elimina una segunda tarea.
4. Recarga la página.
5. Comprueba que únicamente permanecen las tareas que no eliminaste.

CASO 4 · Verificar el identificador

Comprueba que la tarea eliminada corresponde exactamente al botón que presionaste. Si existen varias tareas, no debe eliminarse otra tarjeta por error.

RESULTADO FINAL

Al completar correctamente las pruebas, una tarea puede ser creada, mostrada, eliminada y persistida en localStorage sin afectar las demás tareas.

CHECKLIST DE LA TAREA 6

Antes de entregar, verifica cada punto. Esta lista sirve como control previo, pero la entrega será evaluada de acuerdo con la rúbrica de la actividad.

Interfaz

VERIFICACIÓN	CRITERIO
☐	Cada tarea muestra un botón “Eliminar”.
☐	El botón utiliza la clase delete-button.
☐	El botón se encuentra dentro de la estructura HTML de la tarea.
☐	El botón utiliza correctamente las clases de Bootstrap o los estilos definidos por el proyecto.

TaskManager

VERIFICACIÓN	CRITERIO
☐	Existe el método deleteTask(taskId).
☐	deleteTask() recibe el identificador de la tarea.
☐	Se crea el array newTasks.
☐	Se recorren las tareas existentes.
☐	Se conservan las tareas cuyo id es diferente al taskId.
☐	this.tasks se actualiza con newTasks.

Eventos y persistencia

VERIFICACIÓN	CRITERIO
☐	El EventListener de la Lista de Tareas detecta el botón delete-button.
☐	Se obtiene correctamente la tarea padre.
☐	Se recupera data-task-id.
☐	El taskId se convierte de texto a número.
☐	Se ejecuta taskManager.deleteTask(taskId).
☐	Se ejecuta taskManager.save() después de eliminar.
☐	Se ejecuta taskManager.render() después de eliminar.
☐	La eliminación permanece después de recargar la página.

CIERRE DEL SPRINT 2

IMPORTANTE

La Tarea 6 representa también el cierre del Sprint 2. Por esta razón, la revisión no se limitará a comprobar que la eliminación funciona: se revisará el estado completo del proyecto y la evolución registrada en el repositorio.

1 · REVISIÓN INTEGRAL DEL REPOSITORIO

Al finalizar la Tarea 6, realiza un último commit y asegúrate de que el repositorio contiene los avances correspondientes a todas las tareas desarrolladas durante el Sprint 2, además de conservar los avances del Sprint 1.

- Tarea 4 · Crear la clase TaskManager.
- Tarea 5 · Agregar tareas de manera programada.
- Tarea 6 · Eliminar una tarea.
- Los avances anteriores del proyecto continúan funcionando.
- El historial de commits evidencia la evolución del trabajo durante el Sprint.

NO ENTREGUES ÚNICAMENTE EL RESULTADO FINAL

La revisión del repositorio tendrá en cuenta la trazabilidad del proyecto. Se debe poder identificar la evolución del trabajo mediante los commits realizados durante el Sprint.

2 · REVISIÓN DEL DEPLOY

El proyecto publicado debe reflejar la versión final alcanzada durante el Sprint 2. Antes de entregar, abre el enlace de deploy y prueba la funcionalidad directamente desde la versión publicada.

VERIFICACIÓN	CRITERIO
☐	El deploy corresponde al proyecto actualizado.
☐	La funcionalidad de crear tareas funciona en el deploy.
☐	La funcionalidad de eliminar tareas funciona en el deploy.
☐	La eliminación permanece después de recargar la aplicación publicada.
☐	No existen errores evidentes que impidan utilizar la aplicación.

3 · README.md Y ENLACE DEL DEPLOY

Tal como se solicitó en el cierre del Sprint 1, el README.md será parte de la revisión. Para el cierre del Sprint 2 debes conservar la información relevante y actualizar el enlace de la demo/deploy para que apunte a la versión más reciente del proyecto.

El README debe permitir identificar como mínimo:

- Nombre y descripción breve del Planificador de Tareas.
- Enlace al tablero de trabajo utilizado en el proyecto, si corresponde.
- Enlace al diseño o archivo de Figma utilizado.
- Enlace a la demo/deploy actualizado.
- Cualquier información adicional necesaria para ejecutar o comprender el proyecto.

VERIFICACIÓN DEL ENLACE

No basta con que el enlace exista en el README. Debes abrirlo y comprobar que funciona. El enlace que se utilizará para validar el proyecto será el que esté registrado y actualizado en el README al momento de la entrega.

CHECKLIST DE CIERRE DEL SPRINT 2

VERIFICACIÓN	CRITERIO
☐	El repositorio contiene los avances de las Tareas 4, 5 y 6.
☐	Los avances del Sprint 1 continúan integrados en el proyecto.
☐	El historial de commits evidencia el trabajo realizado.
☐	La aplicación permite crear tareas.
☐	La aplicación permite eliminar tareas.
☐	Las tareas eliminadas no reaparecen al recargar.
☐	El deploy está actualizado con la versión final del Sprint 2.
☐	El enlace del deploy está actualizado en README.md.
☐	El enlace del README fue probado y funciona.
☐	El repositorio está listo para revisión.

ENTREGABLE FINAL

Entrega el enlace del repositorio de GitHub. La revisión utilizará el repositorio como fuente principal para comprobar el código, la evolución del Sprint y el estado final del proyecto. El enlace del deploy será validado desde el README.md.

RESULTADO ESPERADO AL CIERRE DEL SPRINT

Al finalizar esta tarea habrás completado una segunda iteración funcional del Planificador de Tareas. El proyecto deberá permitir registrar tareas, mostrarlas en la interfaz y eliminar aquellas que ya no sean necesarias, conservando los cambios mediante localStorage.

GESTIÓN	INTERACCIÓN	PERSISTENCIA
TaskManager administra la colección de tareas mediante métodos organizados.	La interfaz permite al usuario eliminar una tarea directamente desde su tarjeta.	La eliminación se guarda y se mantiene después de recargar la aplicación.

CIERRE

Antes de entregar, realiza una última revisión completa del proyecto: código, funcionalidad, commits, deploy y README. La Tarea 6 cierra el Sprint 2, por lo que el repositorio debe representar de forma coherente todo el trabajo realizado durante el sprint.